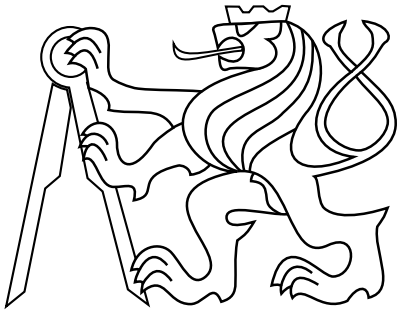
Czech Technical University in Prague

Faculty of Electrical Engineering
Department of Cybernetics
Multi-robot Systems



My Thesis Title
Can Span Multiple Lines

Bachelor's Thesis

John Smith

Prague, January 2021

Study programme: Electrical Engineering and Information Technology
Branch of study: Artificial Intelligence and Biocybernetics

Supervisor: Ing. My Supervisor, Ph.D.

Acknowledgments

Firstly, I would like to express my gratitude to my supervisor.

Swap this for the Thesis Assignment, when you have it from the department.

Declaration

I declare that presented work was developed independently, and that I have listed all sources of information used within, in accordance with the Methodical instructions for observing ethical principles in preparation of university theses.

Date ....

Abstract

The study of autonomous Unmanned Aerial Vehicles (UAVs) has become a prominent sub-field of mobile robotics.

Keywords Unmanned Aerial Vehicles, Automatic Control

Abstrakt

Výzkum na poli autonomních bezpilotních prostředků (UAV) se stal významným oborem mobilní robotiky.

Klíčová slova Bezpilotní Prostředky, Automatické Řízení

Abbreviations

UAV Unmanned Aerial Vehicle

Contents

1 Introduction

First, introduce the reader to the research topic. Start with the most general view and slowly converge to the particular field, sub-field, and the challenges you face. You can cite others' work here **baca2021mrs**.

1.1 Related works

This section should contain related state-of-the-art works and their relation to the author's work. We usually cite the original works like this **benallegue2008high**. You can also cite multiple papers at once like this **baca2016embedded**, **baca2021mrs**.

1.2 Contributions

This section should describe the author's contributions to the field of research.

1.3 Mathematical notation

It is a good practice to define basic mathematical notation in the introduction. See Table 1.1 for an example.

$\mathbf{x}$, $\boldsymbol{\alpha}$	vector, pseudo-vector, or tuple
$\hat{\mathbf{x}}$, $\hat{\boldsymbol{\omega}}$	unit vector or unit pseudo-vector
$\hat{\mathbf{e}}_1, \hat{\mathbf{e}}_2, \hat{\mathbf{e}}_3$	elements of the *standard basis*
$\mathbf{X}, \boldsymbol{\Omega}$	matrix
$\mathbf{I}$	identity matrix
$x = \mathbf{a}^\intercal \mathbf{b}$	inner product of $\mathbf{a}$, $\mathbf{b} \in \mathbb{R}^3$
$\mathbf{x} = \mathbf{a} \times \mathbf{b}$	cross product of $\mathbf{a}$, $\mathbf{b} \in \mathbb{R}^3$
$\mathbf{x} = \mathbf{a} \circ \mathbf{b}$	element-wise product of $\mathbf{a}$, $\mathbf{b} \in \mathbb{R}^3$
$\mathbf{x}_{(n)} = \mathbf{x}^\intercal \hat{\mathbf{e}}_n$	n^{th} vector element (row), $\mathbf{x}, \mathbf{e} \in \mathbb{R}^3$
$\mathbf{X}_{(a,b)}$	matrix element, (row, column)
x_d	x_d is *desired*, a reference
$\dot{x}, \ddot{x}, \dddot{x}, \ddddot{x}$	1^{st}, 2^{nd}, 3^{rd}, and 4^{th} time derivative of x
$x_{[n]}$	x at the sample n
$\mathbf{A}, \mathbf{B}, \mathbf{x}$	LTI system matrix, input matrix and input vector
SO(3)	3D special orthogonal group of rotations
SE(3)	*SO(3)* $\times \mathbb{R}^3$, special Euclidean group

Table 1.1: Mathematical notation, nomenclature and notable symbols.

2 Methodology

2.1 Binary Mask Processing

Binary Mask Creation

A segmentation image is converted into a binary mask by selecting all pixels whose color lies within a specified range. This range can be adjusted by the user to obtain finer control over binary mask creation.

The result is a binary image in which white pixels, referred to as foreground, are assigned the value true, while black pixels, referred to as background, are assigned the value false. We refer to clusters of white or black pixels as regions.

Euclidean Distance Transform

The Euclidean distance transform, referred to as EDT, assigns to each pixel the Euclidean distance to the nearest pixel of a specified set. This can be used for extracting widths from binary masks.

In our implementation, the cost function f is chosen so that distances are computed from foreground pixels to the nearest background pixel. This is determined by which pixels are assigned the values 0 and ∞ in the cost function.

This process can be generalized to transform a cost function on a grid using spatial information [2], such as using distance function other than Euclidean, or n-dimensional grids.

We define cost function as $f(q)$:

$$f(q) = \begin{cases} \infty & \text{if } q \text{ is a foreground pixel,} \\ 0 & \text{otherwise,} \end{cases} \tag{2.1}$$

where q is a pixel of the binary mask. The distance transform of f is denoted by $\mathcal{D}_f$ and defined as

$$\mathcal{D}_f(p) = \min_{q \in \mathcal{G}} \left(d(p,q) + f(q) \right), \tag{2.2}$$

where $\mathcal{G}$ is the grid representing the binary mask, $p \in \mathcal{G}$, and $d(p,q)$ is a distance function.If $d(p,q)$ is chosen as Euclidean distance, then $\mathcal{D}_f$ is called the Euclidean distance transform.

Applying EDT to the binary mask yields a distance field. The value at each element of the distance field is the distance from the corresponding pixel in the binary mask to the nearest background pixel. This can be seen in definition of function $f(q)$, where its value for black pixels is zero, meaning $\mathcal{G}$ computes distance of a white pixel to a black pixel. Consequently, the value of every background pixel in this field is 0.

In our application, this algorithm is used to extract the widths of roads or rivers by computing EDT of foreground pixels. These widths can then be converted to approximate real-world measurements. The precision of this approach depends on the accuracy and resolution of the binary mask.

Signed Distance Field

Signed distance field, or SDF, is a field in which the value corresponding to each pixel of binary mask represents the distance from that pixel to the nearest boundary between foreground and background region. Unlike the unsigned distance field, the SDF stores meaningful values on both sides of the foreground boundary. To differentiate between the two colors, we denote the distances of background pixels as negative.

First we define distance transform $\mathcal{D}_g$ for background pixels by inverting the cost function, so we compute distances of black pixel to a white pixel. The cost function $g(q)$ is

$$g(q) = \begin{cases} \infty & \text{if } q \text{ is a background pixel,} \\ 0 & \text{otherwise,} \end{cases} \tag{2.3}$$

where q is a pixel of the binary mask. The distance transform of g is denoted by $\mathcal{D}_g$ and defined as

$$\mathcal{D}_g(p) = -\min_{q \in \mathcal{G}} \left(d(p,q) + g(q) \right), \tag{2.4}$$

where $\mathcal{G}$ is the grid representing the binary mask, $p \in \mathcal{G}$, and $d(p,q)$ is the Euclidean distance.

Signed distance field has the same dimensions as the initial binary mask. The SDF value v at pixel p is defined as the sum of these two distance transforms

$$v = \mathcal{D}_f(p) + \mathcal{D}_g(p), \tag{2.5}$$

where p is a pixel on binary mask. When p is a foreground pixel, $\mathcal{D}_f(p)$ is positive, while $\mathcal{D}_g(p) = 0$. Conversely, when p is a background pixel, $\mathcal{D}_f(p) = 0$ and $\mathcal{D}_g(p)$ in a negative value.

Binary Mask Smoothing

Binary mask smoothing is achieved by applying Gaussian blur to the computed signed distance field. The blurred field is then classified by a threshold. Values above a this threshold are classified as foreground, while the remaining values are classified as background. When applied to very thin foreground regions, this process may disrupt their continuity.

Gaussian function for two dimensions is defined as

$$G(x,y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2} \right), \tag{2.6}$$

where parameter σ determines the blur strength. Increasing σ results in stronger smoothing, while decreasing produces weaker smoothing.

To blur an image $I(x,y)$ we compute its convolution with Gaussian function

$$I_{\text{blurred}}(x,y) = (I * G)(x,y) = \sum_u \sum_v I(x-u, y-v) G(u,v). \tag{2.7}$$

In discrete implementation we use kernel of size $(2R+1) \times (2R+1)$, where R is a radius generally chosen as

$$R = \max\left(1, \lceil 3\sigma \rceil\right), \tag{2.8}$$

because the value of Gaussian function outside the interval $[-3\sigma, +3\sigma]$ is relatively small.

If we use computed SDF of binary mask as our image, then we get resulting blurred SDF as

$$\mathrm{SDF}_{\mathrm{blurred}}(x, y) = \sum_{u=-R}^{R} \sum_{v=-R}^{R} \mathrm{SDF}(x-u, y-v)G(u, v), \tag{2.9}$$

where $\mathrm{SDF}(x, y)$ is an element of SDF at x and y coordinate, and $0 \leq x <$ width and $0 \leq y <$ height.

The user is given option to adjust both Gaussian blur parameter σ the threshold value. Increasing the threshold erodes the resulting mask, whereas decreasing it expands the foreground region.

2.2 Topology Extraction from Binary Masks

Binary Mask Skeletonization

Thinning of a binary mask, referred to as skeletonization, is a process of removing boundary pixels of a foreground region while maintaining their basic structure, general shape and connectivity. For algorithms used to create binary mask skeleton used for this application, we need to ensure satisfaction of certain criteria:

- The resulting line should be 1 pixel wide with no redundant pixels
- Connectivity of individual components must be preserved
- The skeleton should be approximately centered with respect to the local thickness of the component.
- There should not be substantial gap between skeleton and edge of binary mask, if the foreground region is touching the side.

Many thinning algorithms have been proposed for various applications, one of the popular ones are Guo-Hall algorithm and Zhang-Suen algorithm [3]. Their main advantage is their ability to be parallelized, but both of them do not guarantee desired 1-pixel width without some additional post-processing. A different algorithm was selected [1], which is sequential and guarantees skeleton with no redundant pixels and 1-pixel width.

This algorithm works by performing passes in all four directions and sequentially removes redundant pixels until only the skeleton remains. This results in the skeleton being offset by 1 pixel from center, but this error is insignificant and does not negatively affect the final result.

Additional advantage to its 1-pixel width, is that if foreground region of binary mask is touching image boundary, the algorithm will not create substantial gap between first point of the skeleton and side of the image. For example Zhang-Suen algorithm does not have this feature and it can result in unrealistic rivers or roads.

Disadvantage of skeletonization algorithms is that they are sensitive to binary mask "corners". As the algorithm creates skeleton to be in the middle, when some section of the foreground region is sharper, it can create branch towards this corner. This can be resolved by smoothing the binary mask before performing skeletonization.

Contour Extraction of Area Features

Extracting contours of binary mask foreground region is used for creating features that fill area, such as biomes or lakes.

Used algorithm cycles through every pixel of binary mask. If the pixel is black, that means in background, we skip it. If the pixel is white, in foreground, we count how many 4-neighbors are black and if there are any, we mark this pixel as contour. Pixels that are 4-neighbors are the pixels directly above, below, left or right of the initial pixel.

This algorithm skips foreground pixels along the edges of binary mask, as there are no background pixels. To create edge contour we count pixels that are outside the binary mask bounds as background.

This algorithm, although simple, reliably detects contours. There are a few scenarios where strictly 1-pixel width of the contours is not ensured. These scenarios are hard to resolve by algorithm as their outcome can rapidly change the shape or size of the contours. We decided to leave the resolution of these situations to user by giving them tools for manual edit.

2.3 Polyline Creation from Topological Pixels

A polyline is represented os an ordered list of points $\mathbf{P}_0, \mathbf{P}_1, ..., \mathbf{P}_n$ where each consecutive pair of points $(\mathbf{P}_i, \mathbf{P}_{i+1})$, where $i = 0, 1, ..., n$, defines one line segment. In this way, the polyline forms a connected piecewise curve composed of the segments

$$(\mathbf{P}_0, \mathbf{P}_1), (\mathbf{P}_1, \mathbf{P}_2), ..., (\mathbf{P}_{n-1}, \mathbf{P}_n).$$

Since the points are ordered, the connectivity of the polyline is given implicitly by their order in the list.

Special case of polyline is a closed polyline. A polyline is considered closed if its first and last point are identical,

$$\mathbf{P}_0 = \mathbf{P}_n. \tag{2.10}$$

Pixel Classification

To create polylines we need to classify topological pixels, determining whether a pixel is an endpoint or a junction. We use a simple classification method based on counting the number of foreground pixels in the 8-neighborhood of the examined pixel.

If the number of foreground neighbors is one, the pixel is classified as an endpoint, since it is connected to only one neighboring pixel. If the number is two, the pixel is considered a regular polyline pixel connecting two neighboring pixels.

A junction pixel has three or more foreground neighbors, although in typical cases it has exactly three. If four or more foreground neighbors are present, this may indicate a more complex local structure, such as a plateau or another shape that can complicate polyline extraction.

Polyline Tracing Algorithm

The extracted topology, whether obtained from a binary mask skeleton or from a contour, is still represented as an unordered set of foreground pixels in the image grid. For subsequent processing, such as simplification or spline construction, this discrete representation must be converted into an ordered geometric form.

The goal of the tracing algorithm is therefore to convert the extracted topology pixels into one or more ordered polylines that collectively cover all foreground pixels of the topology.

To achieve this, the algorithm uses the previously classified endpoint and junction pixels together with a visited map. The visited map stores whether a foreground pixel has already been assigned to a traced polyline, which prevents the same branch from being processed repeatedly. Junction pixels require special treatment, because a single junction may belong to multiple output polylines.

The tracing is divided into three steps. First, all branches starting at endpoints are traced until another endpoint or a junction is reached. Second, the remaining branches connecting pairs of junctions are traced. Finally, any still unvisited foreground pixels are assumed to belong to closed contours and are processed separately. This ordering is important, because endpoint-based branches are the least ambiguous, while closed contours can only be identified reliably after all open branches have already been removed.

Tracing from Endpoint to Junction or Endpoint

The first step constructs a polyline starting at an endpoint and following the topology until either another endpoint or a junction is reached. This case corresponds to a branch with a well-defined starting point, and it is therefore the most straightforward tracing scenario.

The algorithm starts by inserting the selected endpoint into the polyline and marking it as visited. It then repeatedly examines the 8-neighborhood of the current pixel and selects the next foreground pixel that has not yet been visited. On the newly found pixel, one of the three following actions is performed:

- If the pixel is an endpoint, it is marked as visited, added to the polyline and the branch is complete.
- If the pixel is an junction, the branch is also terminated, but the pixel is not marked as visited.
- If the pixel is regular topology pixel, it is marked as visited, appended to the polyline, and the tracing continues from that location.

When more than one valid foreground neighbor is available, endpoint or junction candidates are prioritized over regular foreground pixels. Among such candidates, the closest one is selected using Manhattan distance. This additional selection rule improves continuity of the resulting polyline and reduces the occurrence of short redundant segments or incorrect local detours that may otherwise appear in discrete pixel topology.

Algorithm 1: Polyline from Endpoint to Junction or Endpoint algorithm

```
1:  procedure ENDPOINT_TO_JUNCTION_OR_ENDPOINT(starting_endpoint, visited)
2:      queue ← initialize queue
3:      polyline ← initialize empty polyline
4:
5:      queue.add(starting_endpoint)
6:      while queue is not empty do
7:          point ← queue.pop()
8:
9:          visited[ point ] = true
10:         polyline.add(point)
11:
12:         best_endpoint_or_junction ← select from neighbors of point
13:         if best_endpoint_or_junction exists then
14:             polyline.add(best_endpoint_or_junction)
```

```
15:             return polyline
16:         end if
17:
18:         for neighbor from neighbors of point do
19:             if neighbor is foreground and not visited[ neighbor ] then
20:                 queue.add(neighbor)
21:                 break
22:             end if
23:         end for
24:     end while
25:
26:     return polyline
27: end procedure
```

Tracing from Junction to Junction

After all branches with endpoint start vertices have been processed, some foreground pixels may still remain unvisited. In a skeleton topology, these remaining pixels are typically expected to form branches connecting one junction to another. Unlike the previous case, a junction can have multiple outgoing branches, and therefore one starting junction may produce several output polylines.

For this reason, the algorithm considers each unvisited foreground neighbor of the starting junction separately. For every such neighbor, a new polyline is initialized with the starting junction as its first point. If the selected neighbor is itself a junction, the resulting branch consists only of a direct connection between two adjacent junctions, and the tracing for that branch ends immediately.

Otherwise, the neighbor is treated as the first regular pixel of a new branch. The tracing then proceeds in the same way as in the endpoint case. Regular pixels are appended to the polyline and marked as visited, while the first encountered junction terminates the current branch. The starting junction is marked as visited before the traversal begins, but terminal junctions are not marked during branch termination, since they may also participate in other junction-to-junction branches.

As in the previous step, a junction candidate found in the current 8-neighborhood is prioritized over regular foreground pixels. If more such candidates exist, the closest one is selected according to Manhattan distance. This makes the tracing more stable in ambiguous local configurations and helps preserve the intended branch structure of the topology.

Algorithm 2: Polyline from Junction to Junction algorithm

```
1: procedure JUNCTION_TO_JUNCTION(starting_junction, visited)
2:     polyline_list ← initialize empty polyline list
3:
4:     visited[ starting_junction ] = true
5:
6:     for starting_neighbor from neighbors of starting_junction do
7:         if visited[ starting_neighbor ] then
8:             continue
9:         end if
10:
```

```
11:         queue ← initialize queue
12:         polyline ← initialize empty polyline
13:         polyline.add(starting_junction)
14:
15:         queue.add(starting_neighbor)
16:         while queue is not empty do
17:             point ← queue.pop()
18:
19:             if point is junction then
20:                 polyline.add(point)
21:                 break
22:             end if
23:
24:             visited[ point ] = true
25:             polyline.add(point)
26:
27:             best_endpoint_or_junction ← select from neighbors of point
28:             if best_endpoint_or_junction exists then
29:                 polyline.add(best_endpoint_or_junction)
30:                 break
31:             end if
32:
33:             for neighbor from neighbors of point do
34:                 if neighbor is foreground and not visited[ neighbor ] then
35:                     queue.add(neighbor)
36:                     break
37:                 end if
38:             end for
39:         end while
40:
41:         polyline_list.add(polyline)
42:     end for
43:
44:     return polyline_list
45: end procedure
```

Tracing from Closed Contour

After the previous two steps, all open branches are expected to have been removed from consideration. Any remaining unvisited foreground pixels are therefore assumed to belong to closed polylines with no endpoints and no junctions. Such cases occur most commonly when tracing contours of foreground regions rather than skeleton centerlines.

The algorithm begins by selecting any unvisited foreground pixel as the starting point of a new polyline. This point is appended to the polyline, marked as visited, and used as the initial tracing position. The algorithm then repeatedly searches the 8-neighborhood for another unvisited foreground pixel and continues the traversal until no such neighbor exists.

Because this case does not contain endpoints or junctions, no special termination pixel is expected during the traversal. Instead, the tracing ends only when the algorithm returns to

a state in which the current point has no unvisited foreground neighbor. At that moment, the entire closed contour has been traversed. To represent closure explicitly in our implementation, the initial starting point is appended to the polyline once more, so that the first and last point of the resulting ordered list are identical.

Algorithm 3: Polyline from Closed Contour algorithm

```
1:  procedure CLOSED_CONTOUR_TO_POLYLINE(starting_point, visited)
2:      queue ← initialize queue
3:      polyline ← initialize empty polyline
4:
5:      queue.add(starting_point)
6:      while queue is not empty do
7:          point ← queue.pop()
8:
9:          visited[ point ] = true
10:         polyline.add(point)
11:
12:         for neighbor from neighbors of point do
13:             if neighbor is foreground and not visited[ neighbor ] then
14:                 queue.add(neighbor)
15:                 break
16:             end if
17:         end for
18:     end while
19:
20:     polyline.add(starting_point)
21:     return polyline
22: end procedure
```

Edge Cases

There are several edge cases that can be handled by the previously described algorithms, but not in an optimal way. In this application, which also relies heavily on user input, we decided not to handle these cases automatically, as doing so may produce unwanted results. Instead, we provide tools that allow the user to resolve such cases manually, for example by stitching two or more polylines together or by closing a polyline.

Examples of such edge cases include the previously mentioned junction plateaus and L-shaped corners. Although the algorithm can process these cases, it may create redundant line segments or unnecessary connections. Another encountered edge case arises when a 2-pixel-wide contour is created from a foreground region that is too thin. This may result in many short line segments and in a polyline that is not closed.

2.4 Polyline Post-processing

Polyline Stitching Algorithm

Polyline Simplification Using Douglas–Peucker Algorithm

2.5 Spline Construction from Polylines

Hermite and Bézier Curve Representations

In our implementation, we chose to represent splines as a sequence of cubic Bézier curves rather than as Hermite curves, which are used internally by Unreal Engine 5. The main reason for this choice is that Bézier curves are more convenient for the algorithms introduced later in this chapter. At the same time, a cubic Bézier curve can be converted to a Hermite representation in a straightforward manner.

A cubic Bézier curve $\mathbf{B}(t)$ is defined by two end points, $\mathbf{P}_0$ and $\mathbf{P}_3$, and two control points, $\mathbf{P}_1$ and $\mathbf{P}_2$. It is given by

$$\mathbf{B}(t) = (1-t)^3\mathbf{P}_0 + 3(1-t)^2 t\mathbf{P}_1 + 3(1-t)t^2\mathbf{P}_2 + t^3\mathbf{P}_3, \tag{2.11}$$

for $t \in [0, 1]$.

A Hermite curve $\mathbf{H}(t)$ is defined by two end points, $\mathbf{H}_0$ and $\mathbf{H}_1$, and two tangent vectors, $\mathbf{M}_0$ and $\mathbf{M}_1$. It is given by

$$\mathbf{H}(t) = (2t^3 - 3t^2 + 1)\mathbf{H}_0 + (t^3 - 2t^2 + t)\mathbf{M}_0 + (-2t^3 + 3t^2)\mathbf{H}_1 + (t^3 - t^2)\mathbf{M}_1, \tag{2.12}$$

for $t \in [0, 1]$.

A Hermite curve defined by end points $\mathbf{H}_0$, $\mathbf{H}_1$ and tangent vectors $\mathbf{M}_0$, $\mathbf{M}_1$ can be constructed from a cubic Bézier curve defined by control points $\mathbf{P}_0$, $\mathbf{P}_1$, $\mathbf{P}_2$, and $\mathbf{P}_3$ using the following relationships:

$$\mathbf{H}_0 = \mathbf{P}_0, \tag{2.13}$$
$$\mathbf{M}_0 = 3\left(\mathbf{P}_1 - \mathbf{P}_0\right), \tag{2.14}$$
$$\mathbf{M}_1 = 3\left(\mathbf{P}_3 - \mathbf{P}_2\right), \tag{2.15}$$
$$\mathbf{H}_1 = \mathbf{P}_3. \tag{2.16}$$

Catmull–Rom Spline Construction

To interpolate spline through points of a polyline, we employed a Catmull–Rom spline. Catmull–Rom splines are a family of cubic interpolating splines constructed such that a tangent at point $\mathbf{P}_i$ is computed using the previous point $\mathbf{P}_{i-1}$ and the next point $\mathbf{P}_{i+1}$.

We used a class of parameterization described in **bk˙catmull˙rom˙parameterization** of Catmull–Rom spline. Let the knot sequence satisfy

$$t_{k+1} = t_k + \|\mathbf{P}_{k+1} - \mathbf{P}_k\|^\alpha, \tag{2.17}$$

where $\alpha \in [0, 1]$ and $t_0 = 0$. If $\alpha = 0$ then the parameterization is uniform, for $\alpha = 1$ the parameterization becomes chordal.

In our implementation, we use $\alpha = 0.5$, which corresponds to the centripetal parameterization of Catmull–Rom spline. This variant was selected because it is the only parameterization in this family that guarantees no cusps or self-intersections within a single curve segment [4].

As follows from the definition, a Catmull–Rom spline segment is constructed from four points, $\mathbf{P}_0$ to $\mathbf{P}_3$, and produces a curve segment between $\mathbf{P}_1$ and $\mathbf{P}_2$. For an open polyline, this means that the spline would not naturally interpolate the first and the last point of the polyline. To resolve this issue, we introduce auxiliary end points defined as

$$\mathbf{P}_{-1} = 2\mathbf{P}_0 - \mathbf{P}_1, \tag{2.18}$$

$$\mathbf{P}_{n+1} = 2\mathbf{P}_n - \mathbf{P}_{n-1}, \tag{2.19}$$

where the polyline consists of points $\mathbf{P}_0, \ldots, \mathbf{P}_n$.

A drawback of this endpoint extension is that it may produce undesirable shapes when the curve is very sharp near the boundary. However, since the base polylines are constructed from binary mask skeletons or contours, this situation occurs only rarely in practice.

The knot sequence in Equation (2.17) determines the parameter spacing between interpolation points. Unlike the Hermite or Bézier formulations, it does not directly provide a polynomial expression of the curve segment. However, this parameterization can be used to construct equivalent Hermite or Bézier representations of the same curve segment.

The Bézier curve representation of a Catmull–Rom spline segment can be computed directly from the four defining points, $\mathbf{P}_0$ to $\mathbf{P}_3$. If the Bézier curve is defined by control points $\mathbf{B}_0$ to $\mathbf{B}_3$, the relationship is given by [4]

$$\mathbf{B}_0 = \mathbf{P}_1, \tag{2.20}$$

$$\mathbf{B}_1 = \frac{d_1^{2\alpha}\mathbf{P}_2 - d_2^{2\alpha}\mathbf{P}_0 + \left(2d_1^{2\alpha} + 3d_1^{\alpha}d_2^{\alpha} + d_2^{2\alpha}\right)\mathbf{P}_1}{3d_1^{\alpha}\left(d_1^{\alpha} + d_2^{\alpha}\right)}, \tag{2.21}$$

$$\mathbf{B}_2 = \frac{d_3^{2\alpha}\mathbf{P}_1 - d_2^{2\alpha}\mathbf{P}_3 + \left(2d_3^{2\alpha} + 3d_3^{\alpha}d_2^{\alpha} + d_2^{2\alpha}\right)\mathbf{P}_2}{3d_3^{\alpha}\left(d_3^{\alpha} + d_2^{\alpha}\right)}, \tag{2.22}$$

$$\mathbf{B}_3 = \mathbf{P}_2, \tag{2.23}$$

where d_1, d_2, and d_3 are defined as

$$d_1 = \|\mathbf{P}_1 - \mathbf{P}_0\|, \tag{2.24}$$

$$d_2 = \|\mathbf{P}_2 - \mathbf{P}_1\|, \tag{2.25}$$

$$d_3 = \|\mathbf{P}_3 - \mathbf{P}_2\|, \tag{2.26}$$

and $\|\cdot\|$ denotes the Euclidean norm. The knot intervals corresponding to the distances d_1, d_2, and d_3 satisfy

$$t_1 - t_0 = d_1^{\alpha}, \tag{2.27}$$

$$t_2 - t_1 = d_2^{\alpha}, \tag{2.28}$$

$$t_3 - t_2 = d_3^{\alpha}. \tag{2.29}$$

Curve Subdivision Using de Casteljau's Algorithm

Bézier curve can be constructed recursively by using de Casteljau's algorithm. This method is numerically more stable, than using polynomial definition (2.11). Anotherimportant

use case of this algorithm is subdividing Bézier curve into two segments and subsequently getting correct their control points.

Let cubic Bézier curve $\mathbf{B}(t)$ be defined by four control points $\mathbf{P}_0$ to $\mathbf{P}_3$ and $t \in [0, 1]$. De Casteljau's algorithm for cubic Bézier curve $\mathbf{B}$ works by performing three steps. For parameter $u \in [0, 1]$ it creates points

$$\mathbf{Q}_i = (\mathbf{P}_{i+1} - \mathbf{P}_i)u + \mathbf{P}_i, \tag{2.30}$$

for $i = 0, 1, 2$. Point $\mathbf{Q}_i$ is a point on a line connecting control points $\mathbf{P}_i$ an $\mathbf{P}_{i+1}$. Then it creates another points

$$\mathbf{R}_j = (\mathbf{Q}_{j+1} - \mathbf{Q}_j)u + \mathbf{Q}_j, \tag{2.31}$$

for $j = 0, 1$. Similarly as with $\mathbf{Q}_i$, point $\mathbf{R}_j$ is a point on a line connecting points $\mathbf{Q}_j$ and $\mathbf{Q}_{j+1}$. Final point is constructed on a line between $\mathbf{R}_0$ and $\mathbf{R}_1$ as

$$\mathbf{S} = (\mathbf{R}_1 - \mathbf{R}_0)u + \mathbf{R}_0. \tag{2.32}$$

Final point $\mathbf{S}$ lies on a cubic Bézier $\mathbf{B}(t)$ and is equal to a point $\mathbf{B}(u)$.

We can then create two cubic Bézier curves $\mathbf{B}_A(t_A)$ and $\mathbf{B}_B(t_B)$, and $t_A, t_B \in [0, 1]$. Bézier curve $\mathbf{B}_A$ is defined by control points $\mathbf{P}_0$, $\mathbf{Q}_0$, $\mathbf{R}_0$ and $\mathbf{S}$. Bézier curve $\mathbf{B}_B$ is defined by control points $\mathbf{S}$, $\mathbf{R}_1$, $\mathbf{Q}_2$ and $\mathbf{P}_3$. These segments when combined results in the initial Bézier curve $\mathbf{B}(t)$.

3 Conclusion

Summarize the achieved results. Can be similar as an abstract or an introduction, however, it should be written in past tense.

4 References

[1] H. Jain and A. P. Kumar, *A Sequential Thinning Algorithm For Multi-Dimensional Binary Patterns*, en, Oct. 2017. Accessed: Mar. 29, 2026. [Online]. Available: https://arxiv.org/abs/1710.03025v2

[2] P. F. Felzenszwalb and D. P. Huttenlocher, "Distance Transforms of Sampled Functions," *Theory of Computing*, vol. 8, pp. 415–428, Sep. 2012, Number: 19. DOI: 10.4086/toc.2012.v008a019 Accessed: Mar. 29, 2026. [Online]. Available: https://theoryofcomputing.org/articles/v008a019/

[3] *A fast parallel algorithm for thinning digital patterns | Communications of the ACM*. Accessed: Mar. 29, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/357994.358023

[4] *On the parameterization of Catmull-Rom curves | 2009 SIAM/ACM Joint Conference on Geometric and Physical Modeling*. Accessed: Mar. 29, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/1629255.1629262

A Appendix A